1.PySpark SQL

PySpark SQL Tutorial – The `pyspark.sql` is a module in PySpark that is used to perform SQL-like operations on the data stored in memory. You can either leverage using programming API to query the data or use the ANSI SQL queries similar to RDBMS. You can also mix both, for example, use API on the result of an SQL query.
Following are the important classes from the SQL module.

- pyspark.sql.SparkSession – SparkSession is the main entry point for DataFrame and SQL functionality. It is responsible for coordinating the execution of SQL queries and DataFrame operations. `SparkSession` can be created using the `SparkSession.builder` API. It encapsulates the functionality of the older `SQLContext` and `HiveContext`.
- pyspark.sql.DataFrame – DataFrame is a distributed collection of data organized into named columns. DataFrames can be created from various sources like CSV, JSON, Parquet, Hive, etc., and they can be transformed using a rich set of high-level operations.
- pyspark.sql.Column – A column expression in a DataFrame.It can be used to reference, manipulate, and transform columns.
- pyspark.sql.Row – A row of data in a DataFrame.Rows are used to store and manipulate data in a distributed and structured way. Each `Row` object can be thought of as a record or a tuple with named fields, similar to a row in a relational database table.
- pyspark.sql.GroupedData – An object type returned by DataFrame.groupBy() This class provides methods for calculating summary statistics, aggregating data, and applying various functions to grouped data.
- `pyspark.sql.DataFrameNaFunctions` – Methods for handling missing data (null values). This class is specifically designed to handle operations related to missing data and provides functionalities for filling, dropping, and replacing null values in a PySpark DataFrame.
- `pyspark.sql.DataFrameStatFunctions` – This class is part of the PySpark SQL module and is designed to facilitate the computation of summary statistics on numerical columns in a DataFrame. It offers methods for calculating various descriptive statistics, correlation, covariance, and more.
- pyspark.sql.functions – List of standard built-in functions.
- pyspark.sql.types – Available SQL data types in PySpark.
- pyspark.sql.Window – Would be used to work with window functions.

Regardless of what approach you use, you have to create a SparkSession which is an entry point to the PySpark application.

```
# Import SparkSession
from pyspark.sql import SparkSession

# Create SparkSession
spark = SparkSession.builder \
      .master("local[1]") \
      .appName("SparkByExamples.com") \
      .getOrCreate()
```

2. PySpark SQL DataFrame API

The PySpark SQL DataFrame API provides a high-level abstraction for working with structured and tabular data in PySpark. It offers functionalities to manipulate, transform, and analyze data using a DataFrame-based interface. Here's an overview of the PySpark SQL DataFrame API:

1. **DataFrame Creation**: DataFrames can be created from various data sources such as CSV files, JSON files, Parquet files, Hive tables, SQL queries, RDDs, or Python collections like lists and dictionaries.
2. **Data Manipulation**: DataFrame API provides a rich set of methods for data manipulation including filtering, selecting, aggregating, joining, grouping, sorting, and windowing operations. These operations can be chained together to perform complex data transformations.
3. **Column Operations**: DataFrame API allows operations on individual columns such as renaming columns, adding new columns, dropping columns, and applying functions to columns.
4. **Data Types and Schemas**: DataFrames have a schema that defines the structure of the data including column names and data types. DataFrame API provides methods to infer or specify schemas, and to access and modify schema information.
5. **Interoperability with SQL**: DataFrames can be registered as temporary or persistent views, allowing seamless interoperability with Spark SQL. SQL queries can be executed directly on DataFrame views using SparkSession's SQL engine.
6. **Optimizations**: DataFrame API includes optimizations such as predicate pushdown, column pruning, and code generation to improve query performance and execution efficiency.
7. **Integration with Machine Learning**: PySpark's DataFrame API integrates seamlessly with Spark's MLlib library, enabling machine learning tasks such as feature engineering, model training, and prediction using DataFrames.

8. **Parallel Processing**: DataFrame operations are designed to leverage Spark's distributed computing capabilities, allowing data processing to be performed in parallel across a cluster of machines.

3. Benefits of using SQL Queries

PySpark enables running SQL queries through its SQL module, which integrates with Spark's SQL engine. SQL is a widely used language for querying and manipulating data in relational databases. By using SQL queries in PySpark, users who are familiar with SQL can leverage their existing knowledge and skills to work with Spark DataFrames.

SQL provides a concise and intuitive syntax for expressing data manipulation operations such as filtering, aggregating, joining, and sorting. Writing SQL queries can often be more readable and maintainable compared to equivalent DataFrame API code.

Spark's SQL engine includes an advanced query optimizer that can optimize SQL queries for better performance. The optimizer can perform optimizations such as predicate pushdown, join reordering, and column pruning to improve query execution speed.

Finally, PySpark seamlessly integrates SQL queries with DataFrame operations. Users can mix and match SQL queries with DataFrame API calls within the same PySpark application, providing flexibility and interoperability.

4. PySpark SQL Examples

Running SQL-like queries in PySpark involves several steps. Below are the step-by-step instructions:

- Start by initializing a SparkSession. This is the entry point to PySpark.
- Load your data into a DataFrame. You can read data from various sources such as CSV files, Parquet files, or JDBC data sources.
- Register the DataFrame as a temporary view so that you can query it using SQL.
- Now you can run SQL queries on the registered temporary view using the `spark.sql()` method.
- Process the results returned by the SQL query. You can further manipulate the results as needed.

4.1 Create SQL Temporary View or Table

When you create a temporary table in PySpark, you're essentially registering a DataFrame as a temporary view. This allows you to query the DataFrame using SQL syntax through SparkSession's SQL engine. The temporary table is scoped to the SparkSession in which it was created. It is only available for the duration of that session and does not persist across sessions or applications.

Create a DataFrame from a CSV file. You can find this CSV file at Github project.

```
# Read CSV file into table
df = spark.read.option("header",True) \
            .csv("/Users/admin/simple-zipcodes.csv")
df.printSchema()
df.show()
```

To use an ANSI SQL query similar to RDBMS, create a temporary table by reading the data from a CSV file. You can find this CSV file at Github project.

```
# Create temporary table
spark.read.option("header",True) \
            .csv("/Users/admin/simple-zipcodes.csv") \
            .createOrReplaceTempView("Zipcodes")
```

Yields below output.

```
root
 |-- RecordNumber: string (nullable = true)
 |-- Country: string (nullable = true)
 |-- City: string (nullable = true)
 |-- Zipcode: string (nullable = true)
 |-- State: string (nullable = true)

+------------+-------+--------------------+-------+-----+
|RecordNumber|Country|                City|Zipcode|State|
+------------+-------+--------------------+-------+-----+
|           1|     US|          PARC PARQUE|    704|   PR|
|           2|     US|PASEO COSTA DEL SUR|    704|   PR|
|          10|     US|         BDA SAN LUIS|    709|   PR|
|       49347|     US|                HOLT|  32564|   FL|
|       49348|     US|            HOMOSASSA|  34487|   FL|
|       61391|     US|   CINGULAR WIRELESS|  76166|   TX|
|       61392|     US|           FORT WORTH|  76177|   TX|
|       61393|     US|             FT WORTH|  76177|   TX|
|       54356|     US|          SPRUCE PINE|  35585|   AL|
|       76511|     US|             ASH HILL|  27007|   NC|
|           4|     US|     URB EUGENE RICE|    704|   PR|
|       39827|     US|                MESA|  85209|   AZ|
|       39828|     US|                MESA|  85210|   AZ|
|       49345|     US|            HILLIARD|  32046|   FL|
|       49346|     US|              HOLDER|  34445|   FL|
|           3|     US|       SECT LANAUSSE|    704|   PR|
|       54354|     US|       SPRING GARDEN|  36275|   AL|
|       54355|     US|         SPRINGVILLE|  35146|   AL|
|       76512|     US|            ASHEBORO|  27203|   NC|
|       76513|     US|            ASHEBORO|  27204|   NC|
+------------+-------+--------------------+-------+-----+
```

4.2 PySpark SQL to Select Columns

The select() function of DataFrame API is used to select the specific columns from the DataFrame.

```
# DataFrame API Select query
df.select("country","city","zipcode","state") \
      .show(5)
```

In SQL, you can achieve the same using `SELECT FROM` clause as shown below.

```
# SQL Select query
spark.sql("SELECT country, city, zipcode, state FROM ZIPCODES") \
      .show(5)
```

Both above examples yield the below output.

```
+-------+--------------------+-------+-----+
|country|                city|zipcode|state|
+-------+--------------------+-------+-----+
|     US|          PARC PARQUE|    704|   PR|
|     US|PASEO COSTA DEL SUR|    704|   PR|
|     US|        BDA SAN LUIS|    709|   PR|
|     US|                HOLT|  32564|   FL|
|     US|           HOMOSASSA|  34487|   FL|
+-------+--------------------+-------+-----+
only showing top 5 rows
```

4.3 Filter Rows

To filter the rows from the data, you can use where() function from the DataFrame API.

```
# DataFrame API where()
df.select("country","city","zipcode","state") \
   .where("state == 'AZ'") \
   .show(5)
```

Similarly, in SQL, you can use `WHERE` clause as follows.

```
# SQL where
spark.sql(""" SELECT  country, city, zipcode, state FROM ZIPCODES
       WHERE state = 'AZ' """) \
        .show(5)
```

Yields below output.

```
+-------+----+-------+-----+
|country|city|zipcode|state|
+-------+----+-------+-----+
|     US|MESA|  85209|   AZ|
|     US|MESA|  85210|   AZ|
+-------+----+-------+-----+
```

4.4 Sorting

To sort rows on a specific column, use orderBy() function on DataFrame API.

```
# sorting
df.select("country","city","zipcode","state")  \
   .where("state in ('PR','AZ','FL')")  \
   .orderBy("state")  \
   .show(10)
```

In SQL, you can achieve sorting by using `ORDER BY` clause.

```
# SQL ORDER BY
spark.sql(""" SELECT  country, city, zipcode, state FROM ZIPCODES
       WHERE state in ('PR','AZ','FL') order by state """)  \
        .show(10)
```

4.5 Grouping

The groupBy().count() is used to perform the group by on DataFrame.

```
# grouping
df.groupBy("state").count()  \
   .show()
```

You can achieve group by in PySpark SQL is by using `GROUP BY` clause.

```
# SQL GROUP BY clause
spark.sql(""" SELECT state, count(*) as count FROM ZIPCODES
       GROUP BY state""")  \
        .show()
```

4.6 SQL Join Operations

Similarly, if you have two tables, you can perform the Join operations in PySpark.

Complete Example

```
# Import
from pyspark.sql import SparkSession

# Create SparkSession
spark = SparkSession.builder.appName('SparkByExamples.com') \
                     .getOrCreate()

# Create DataFrame
df = spark.read \
          .option("header",True) \
          .csv("/Users/admin/simple-zipcodes.csv")
df.printSchema()
df.show()

# Create SQL table
spark.read \
          .option("header",True) \
          .csv("/Users/admin/simple-zipcodes.csv") \
          .createOrReplaceTempView("Zipcodes")

# Select query
df.select("country","city","zipcode","state") \
      .show(5)

spark.sql("SELECT  country, city, zipcode, state FROM ZIPCODES") \
      .show(5)

# where
df.select("country","city","zipcode","state") \
   .where("state == 'AZ'") \
   .show(5)

spark.sql(""" SELECT  country, city, zipcode, state FROM ZIPCODES
      WHERE state = 'AZ' """) \
       .show(5)

# sorting
df.select("country","city","zipcode","state") \
   .where("state in ('PR','AZ','FL')") \
   .orderBy("state") \
   .show(10)

spark.sql(""" SELECT  country, city, zipcode, state FROM ZIPCODES
      WHERE state in ('PR','AZ','FL') order by state """) \
       .show(10)

# grouping
df.groupBy("state").count() \
   .show()
```

```
spark.sql(""" SELECT state, count(*) as count FROM ZIPCODES
      GROUP BY state""")  \
       .show()
```